

RL-Based Adaptive Data Reduction for IoV Digital Twin Synchronization

Rashmi Mudli Gowda

Research Paper Project 5344

Abstract—Vehicular Digital Twins require frequent updates, but continuously transmitting sensor data can consume limited V2X bandwidth. This project presents a simulation-based framework that combines OFDM-style ISAC processing, Q-learning, and Digital Twin feedback. Vehicle, network, sensing, and synchronization information form a seven-feature state used to select transmit, delay, or drop. On a synthetic trace, Q-learning reduced transmitted data by 54.5% compared with always transmitting, while Digital Twin error increased from 0.364 m to 0.460 m. Periodic scheduling performed slightly better in this small experiment. The results demonstrate the complete workflow and the trade-off between communication usage and Digital Twin accuracy, but further testing with larger datasets and real ISAC measurements is required.

I. INTRODUCTION

The Internet of Vehicles (IoV) allows vehicles to communicate with other vehicles, roadside units, edge servers, and cloud systems. A Digital Twin is a virtual representation of a physical vehicle that can be used for monitoring, prediction, and analysis. To remain accurate, the Digital Twin must receive regular updates from the vehicle. However, continuously sending data from cameras, LiDAR, radar, and other sensors can consume significant network bandwidth. Fixed transmission schedules are also inefficient because they do not respond to changing vehicle behavior or network conditions.

This project examines whether reinforcement learning can help decide which vehicle updates should be transmitted. Instead of sending complete radio waveforms or every sensor record, the proposed ISAC module processes the signal locally and extracts compact information such as estimated range, radial velocity, sensing confidence, and observation change. This information is combined with vehicle speed and acceleration, network latency and bandwidth, and Digital Twin synchronization error to form the state given to a Q-learning agent.

The agent acts as a communication scheduler rather than a driving controller. For each non-safety-critical update, it chooses whether to transmit the update immediately, delay it temporarily, or drop it. The Digital Twin then predicts or corrects its state and returns synchronization error as feedback. The current work implements this complete workflow using a simulated OFDM-style ISAC response, a seven-feature RL state, a predictive Digital Twin, and comparisons with periodic and always-transmit methods.

II. RELATED WORK

Prior work has studied digital-twin synchronization using optimization, value iteration, SARSA, Q-learning, and wireless resource scheduling [2]. Existing approaches often optimize device-level scheduling or resource allocation. The present project focuses on fine-grained vehicle-data selection: deciding which update to transmit, delay, or discard under changing mobility and network conditions. This complements ISAC architectures in which sensing and communication share constrained resources.

III. SYSTEM MODEL

A. Layered Architecture

The simulated RSU transmits a pilot waveform, extracts range-Doppler features from the reflected response, and combines them with vehicle and network variables. A V2X link carries only selected compact updates to the edge. The edge-side Digital Twin predicts range and radial velocity, applies received corrections, and returns synchronization error to the vehicle-side scheduler [1].

Element	Implementation
State	Speed, acceleration, latency, bandwidth, DT error, ISAC confidence, innovation
Actions	Transmit, delay, drop
Agent	Tabular Q-learning
Baselines	Periodic / 3 and always transmit

B. Simulated ISAC Sensing Front End

The available datasets do not contain real channel-state information or radar echoes. Therefore, the project generates a simplified OFDM-style ISAC response that represents how a signal could reflect from a moving vehicle. The simulation models two important effects: signal delay, which represents the distance between the vehicle and RSU, and Doppler shift, which represents vehicle motion toward or away from the RSU. Controlled noise is added to make the response more realistic. The simulation uses a 5.9-GHz carrier, 100-MHz bandwidth, 64 subcarriers, 32 time symbols, and a 0.25-ms symbol interval. These are representative simulation settings rather than measurements from real ISAC hardware.

The simulated response is processed across frequency and time to create a range-Doppler map. The strongest point in this map provides the estimated vehicle range and radial velocity. Its strength compared with the rest of the map is used to calculate sensing confidence. The change between consecutive range and velocity estimates is used to calculate innovation, which indicates how much new information is present in the latest observation. The complete waveform remains at the local processor, while only compact sensing information is provided to the communication scheduler.

C. State Formation

At each time step, the system creates an RL state using seven features: vehicle speed, acceleration, network latency, available bandwidth, Digital Twin synchronization error, ISAC confidence, and innovation. Together, these features describe the vehicle's movement, current network condition, reliability of the sensing observation, importance of the latest change, and accuracy of the Digital Twin.

Since tabular Q-learning requires a limited number of states, each continuous value is divided into predefined ranges. These combinations produce 11,520 possible states. Estimated range and radial velocity are used directly to predict and correct the Digital Twin instead of being added as separate Q-table features. Their change is represented through innovation, while their difference from the Digital Twin estimate is represented through synchronization error.

For every state, the Q-learning agent selects one of three communication actions: transmit, delay, or drop. The reward considers Digital Twin error, bandwidth usage, communication latency, and an additional penalty for delaying or dropping an update. Digital Twin accuracy receives the greatest weight [2]. As a result, the agent is encouraged to reduce unnecessary communication while preventing the Digital Twin from becoming too inaccurate.

D. Q-learning of the Three Actions

TRANSMIT sends the current compact sensing update. Link quality is modeled from bandwidth and latency, and the Digital Twin correction gain is additionally scaled by ISAC confidence. A clear observation on a good link therefore corrects range and velocity more strongly than a weak observation on a poor link.

DELAY stores the newest pending payload size without immediately correcting the twin. A later delayed observation supersedes the older one, which prevents stale packets from accumulating. The Digital Twin continues predicting its range using its current velocity until a transmission is selected.

DROP permanently discards the current update. No payload is buffered and the Digital Twin continues open-loop prediction. The action is appropriate only when the observation adds little information and synchronization error remains acceptable.

E. Current Buffering Limitation

In the current proof of concept, selecting **delay** stores only the size of the pending update, represented by `buffered_payload_kb`. It does not store the complete sensing record, such as the timestamp, estimated range, velocity, confidence, and event type. Therefore, the current buffer represents communication cost rather than a complete real-world packet queue.

Packet loss is also not modeled directly. Its effect is represented indirectly through changing bandwidth and latency. A real implementation should explicitly model packet loss, queue length, update age, retransmissions, and packet expiration. It should also preserve complete timestamped updates and ensure that safety-critical events, such as sudden braking, bypass the RL scheduler and are transmitted immediately.

IV. Q-LEARNING METHOD

The Q-learning agent learns which communication action is most suitable for each state. It maintains a table, $Q(s,a)$, containing a value for every state–action combination. These values are initially set to zero because the agent has not yet learned which actions are useful.

During training, the agent uses an epsilon-greedy strategy. At the beginning, it frequently selects random actions to explore the effects of transmit, delay, and drop. As training continues, the exploration rate decreases, and the agent increasingly selects the action with the highest learned Q-value.

After every action, the agent receives a reward and observes the next state. It then updates the selected Q-value using the immediate reward and the expected value of future actions. The implementation uses a learning rate of 0.15, a discount factor of 0.95, and 150 training episodes. Epsilon decreases from 1.0 to 0.05, while random seed 5433 is used to make the experiment reproducible.

A. Why Q-Learning Was Selected

Q-learning was selected as a simple and understandable baseline for testing the proposed ISAC-RL-Digital Twin workflow. After discretization, the seven input features create 11,520 possible states and 34,560 state–action values. This is manageable with a Q-table and allows us to inspect what the agent learns for every state and action.

The available data does not contain labels indicating whether an update should be transmitted, delayed, or dropped. Therefore, a supervised-learning model would require manually created target labels. Q-learning does not need those labels. Instead, it learns through a reward that considers Digital Twin accuracy and communication cost.

The current objective is to verify that the complete pipeline works and demonstrates the trade-off between reducing communication and maintaining Digital Twin synchronization. Q-learning provides this verification without introducing unnecessary model complexity. DQN is planned for longer traces and continuous, higher-dimensional states. PPO will be considered when the actions include continuous decisions such as transmission power, data rate, compression level, or waiting time.

B. Why Other Algorithms Are Deferred

A DQN replaces the Q-table with a neural approximator and is appropriate for longer traces, continuous inputs, and richer temporal context while retaining discrete actions. PPO is better suited to continuous or hybrid control. Neither method is justified by a synthetic record trace: both would add tuning complexity without credible generalization evidence.

V. DATASETS AND PREPROCESSING

The project contains three input datasets, but the current reported experiment uses only `synthetic_data.csv`. This data representing approximately one second of vehicle and network activity. Each record includes vehicle speed, acceleration, heading, network latency, available bandwidth, and a Digital Twin error value.

During preprocessing, the loader converts the data into a common format and assigns a simulated 64-KB payload to each update. Since the dataset does not contain real ISAC radio measurements, the ISAC module generates a simulated response and derives estimated range, radial velocity, an SNR proxy, sensing confidence, innovation, and estimation errors.

The loader also supports `Dataset.csv` and the NGSIM trajectory dataset. so the current sensing results are simulation-based [6].

VI. EXPERIMENTAL SETUP

The experiment trains the Q-learning agent by synthetic data, trace for 150 episodes. During training, the agent explores the transmit, delay, and drop actions and updates its Q-table using the resulting Digital Twin error and communication cost. After training, the learned policy is evaluated once by selecting the action with the highest Q-value.

The learned policy is compared with two baseline methods: transmitting every update and transmitting periodically at every third record. Performance is measured using Digital Twin synchronization error, transmitted data volume, transmission rate, and total reward. The simulated ISAC module is evaluated using sensing confidence, range error, and radial-velocity error [3].

This experiment is intended to verify that the complete ISAC-RL-Digital Twin pipeline operates correctly and exposes the communication accuracy trade-off. Because the same short trace is used during training and evaluation, the results do not demonstrate performance on unseen data or real-world generalization.

VII. PRELIMINARY RESULTS

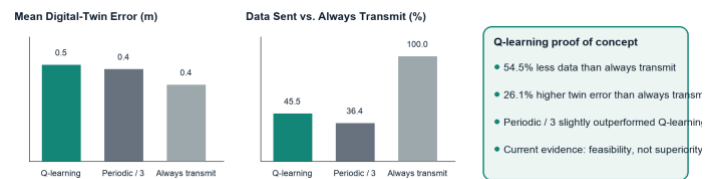


Fig. 1. Verified synthetic policy trade-off and simulated ISAC estimation errors.

The Q-learning policy transmitted 0.3125 MB of data, which is 54.5% less than the 0.6875 MB used by the always-transmit method. This reduction in communication came with a higher mean Digital Twin error, which increased from 0.364 m to 0.460 m [5].

The periodic method transmitted 0.250 MB and achieved a mean error of 0.439 m. Therefore, it performed slightly better than Q-learning in this small experiment. These results do not show that Q-learning is currently the best scheduling method. Instead, they demonstrate the trade-off between reducing network usage and maintaining Digital Twin accuracy.

The simulated ISAC module achieved a mean range error of 0.371 m, a mean radial-velocity error of 0.767 m/s, and a mean confidence score of 0.998. These results verify the simulated sensing and communication pipeline but do not represent performance from real ISAC hardware.

VIII. LIMITATIONS

All waveform and sensing results in this project are generated through simulation rather than measured from real ISAC hardware. The SNR value is estimated from bandwidth and latency for simulation purposes and does not represent an actual radio-channel measurement. Because the dataset does not provide RSU coordinates, the vehicle is assumed to move directly toward or away from the RSU.

The experiment also uses only synthetic records, with the same trace used for both training and evaluation. In addition, packet loss and queue occupancy are not explicitly included in the RL state, and the sensing-confidence score only represents the strength of the detected peak rather than a calibrated probability.

Therefore, the current results verify that the proposed pipeline can be implemented, but they do not demonstrate real-world ISAC accuracy, real-time network performance, generalization to unseen data, or superiority over conventional scheduling methods.

IX. CONCLUSION AND FUTURE WORK

This project presents a simulation-based ISAC-RL-Digital Twin communication framework. The simulated ISAC module extracts compact information such as range, radial velocity, confidence, and innovation. Q-learning then decides whether a non-safety-critical update should be transmitted, delayed, or dropped. The edge Digital Twin predicts and corrects the vehicle state and returns synchronization error as feedback. The current implementation demonstrates that the complete workflow can operate in a reproducible simulation.

Future work will use measured ISAC or radar data with known vehicle and RSU locations. The network model will include packet loss, queues, congestion, and multiple vehicles competing for bandwidth. Longer datasets, separate training and testing traces, and confidence intervals will also be required. DQN will be evaluated for larger continuous states, while PPO will be considered when communication actions include continuous transmission rate, power, compression, or waiting time. These improvements are necessary before evaluating the approach for real-world deployment.

REFERENCES

- [1]<https://www.sciencedirect.com/science/article/pii/S2590174526004939>
- [2] <https://www.mdpi.com/2079-9292/14/17/3387>
- [3] <https://arxiv.org/abs/2511.05789>
- [4] <https://ieeexplore.ieee.org/document/9759989>
- [5] <https://ieeexplore.ieee.org/document/10906634>
- [6]<https://www.sciencedirect.com/science/article/pii/S0169023X24000284>
- [7] <https://pmc.ncbi.nlm.nih.gov/articles/PMC12989590/>